DETERMINISTIC NETWORK OPERATING SYSTEM (DNOS)

Security + Mempool + Canonical Storage + Reconciliation Engine + Recovery Governance Layer

Version: 1.0 — Unified Production Specification (Complete)

---

1. SYSTEM SCOPE

DNOS is a deterministic network operating system designed for adversarial environments where:

- Every event is cryptographically verifiable
- Every state is reproducible via snapshot execution
- Every decision is deterministic and policy-bound
- All storage is canonical and content-addressed
- All inconsistencies are structurally detectable and recoverable
- No runtime behavior depends on non-deterministic external state

---

2. CORE SYSTEM EQUATION

```id="eq1"
f(E, S, M, R, W) → D
```

Where:

E = network event

S = canonical storage state

M = HD mempool projection

R = reconciliation state

W = immutable genesis policy weights

D = deterministic decision output

---

3. CANONICAL STORAGE LAYER

3.1 Definition

Immutable content-addressed storage layer.

3.2 Canonicalization Pipeline

content → canonical form

canonical form → hash

hash → storage address

3.3 Core Property

storage(content) = f(canonical(content))

3.4 Properties

deterministic addressing

immutability per version

reproducibility

implementation independence

---

4. HD MEMPOOL LAYER

4.1 Definition

The mempool is a deterministic projection, NOT a source of truth.

4.2 Structure

```
type HDMempoolEntry = {
  eventHash: string;
  canonicalHash: string;
  storagePointer: string;
  previousHash: string;
  state: "pending" | "confirmed" | "rejected";
  snapshotId: string;
};
```

4.3 Core Relation

Mempool = projection(Storage, Snapshot)

---

5. SNAPSHOT ISOLATION LAYER

5.1 Definition

All execution is bound to frozen system state.

```
type SystemSnapshot = {
  snapshotId: string;
  mempoolRootHash: string;
  storageRootHash: string;
  reconciliationRootHash: string;
  weightVectorHash: string;
  timestamp: string;
};
```

5.2 Isolation Rule

decision(event) MUST use snapshot(t0)
NOT live state

5.3 Effect

eliminates race conditions

prevents replay drift

guarantees deterministic evaluation

---

6. SECURITY KERNEL

6.1 Attack Model

```
type AttackClass =
  | "REPLAY"
  | "SIGNATURE_INVALID"
  | "PAYLOAD_TAMPER"
  | "STATE_DESYNC"
  | "DDoS_PATTERN"
  | "ANOMALY_BEHAVIOR";
```

6.2 Threat Scoring

```
score = f(ip, asn, entropy, burst, session)
```

6.3 DDoS Detection

```
if (
  rps > threshold &&
  asnSpread > k &&
  entropyVariance > v
) => DDoS_PATTERN
```

---

7. REPLAY ENGINE

7.1 Structure

type ReplayStore = Set<string>;

7.2 Rule

if eventHash ∈ ReplayStore → REJECT
else → ACCEPT + store

7.3 Cross-validation

Replay validated against:

storage hash

mempool hash

snapshot hash

---

8. RECONCILIATION ENGINE

8.1 Definition

Ensures coherence between:

storage

mempool

external evidence

8.2 Consistency Rule

storage_hash == mempool_hash == reconciliation_hash

If violated:

STATE = DESYNC

8.3 Output Model

```
type ReconciliationResult = {
  valid: boolean;
  divergencePoint?: string;
  affectedLayer: "storage" | "mempool" | "external";
};
```

---

9. CONSENSUS SCORE FUNCTION (CSF)

9.1 Definition

score(event) = Σ (w_i * feature_i)

9.2 Normalization Constraint

Σ w_i = 1

9.3 Genesis Invariance

W = W_genesis (immutable)

9.4 Final Form

```
score(event) =
  Σ (w_i(genesis) * feature_i(event, snapshot))
```

---

10. CROSS-LAYER VALIDATION PIPELINE

1. Event ingestion

2. Signature verification

3. Replay detection

4. Snapshot binding

5. Attack classification

6. Mempool projection

7. Canonical storage write

8. Reconciliation validation

9. Decision execution

---

11. SECURITY KERNEL EXECUTION MODEL

```
function deterministicSecurityOS(event, context, snapshot) {

  if (!verifyEvent(event)) {
    return { action: "REJECT", reason: "SIGNATURE_INVALID" };
  }

  if (detectReplay(event.eventId)) {
    return { action: "BLOCK", reason: "REPLAY_ATTACK" };
  }

  const ctx = bindSnapshot(context, snapshot);
```

```
  const attackClass = classify(event, ctx);

  if (attackClass === "DDoS_PATTERN") {
    return { action: "DROP", reason: attackClass };
  }

  const mempoolEntry = projectToMempool(event, snapshot);

  const storagePointer = canonicalWrite(event);

  const sync = reconcile(mempoolEntry, storagePointer);

  if (!sync.valid) {
    return { action: "QUARANTINE", reason: "STATE_DESYNC" };
  }

  return {
    action: "ALLOW",
    mempoolEntry,
    storagePointer
  };
}
```

---

12. QUARANTINE & RECOVERY ENGINE

12.1 Trigger

DESYNC → QUARANTINE

12.2 Recovery Principle

> reconstruct maximal consistent subgraph under snapshot isolation

---

12.3 Complexity Control Layer (CRITICAL)

Structural constraints:

partitioning of graph ($G \rightarrow G_1..Gk$)

canonical prefix pruning

greedy selection (no global optimization)

bounded depth d_max (genesis constant)

consistency cache reuse

early termination threshold τ

12.4 Feasible State Space

$\Omega_{restricted} = valid(snapshot\_locality \wedge hash\_consistency \wedge depth \leq d_{max})$

12.5 Optimized Recovery Objective

$C^* = \arg\max_{C \in \Omega_{restricted}} consistency(C)$

12.6 Complexity Bound

$O(k \cdot n \log n)$

12.7 Exit Condition

storage == mempool == reconciliation → EXIT QUARANTINE

---

13. RECOVERY GOVERNANCE LAYER (RCGL)

13.1 Purpose

Enforces bounded execution of recovery.

13.2 Invariants

bounded execution time

bounded search space

adversarial containment locality

13.3 RCGL Function

RCGL = enforce(partition_boundaries, depth_limit, cache, early_stop)

---

14. SYSTEM PROPERTIES

global determinism

snapshot-bound execution

structural anti-replay

Byzantine inconsistency detection (non-voting)

deterministic recovery

bounded complexity guarantees

full auditability of attack states

---

15. ARCHITECTURAL DEFINITION

DNOS is not a set of subsystems.

It is:

> a deterministic adversarial operating system with cryptographically enforced cross-layer state coherence under immutable genesis policy execution

---

16. FINAL SYSTEM INVARIANT

S ≡ M ≡ R

Under constraint:

> all transitions are snapshot-bound functions over canonical storage and immutable genesis policy vector

---

17. KEY ARCHITECTURAL INSIGHT

No runtime global optimization exists

All “complexity problems” are structurally prevented

Recovery is not computation of truth, but reconstruction of bounded consistency space

---

18. FINAL GUARANTEE STATEMENT

DNOS guarantees:

> Any system divergence is transformed into a bounded deterministic reconstruction problem with polynomial-time recovery under strict structural constraints defined at genesis.